Self-managed API 0.1.0

SDK, CLI y worker local para mantener integraciones de API.

El proyecto funciona en el entorno de quien lo instala, sin cuenta del proyecto, token de licencia ni servidor de control. El código usa licencia MIT. Los modelos, GitHub y las fuentes remotas son integraciones opcionales, configuradas y autorizadas por el usuario.

Contratos y código local → comparación e impacto
→ propuesta → revisión humana → aprobación de tests
→ original y candidato aislados → aprobación de publicación
→ pull request en borrador → revisión y merge del usuario

Primer recorrido sin servidor ni modelo

Requisitos: Node 24.21.0-24.x y pnpm 11.19.0. La instalación descarga dependencias. Los ejemplos siguientes leen fixtures ficticias y no contactan servicios externos.

```
git clone https://github.com/Altitud-Studio-S-L-U/self-managed-api.git
cd self-managed-api
pnpm install --frozen-lockfile
pnpm build
pnpm self-managed-api --help
pnpm --silent self-managed-api mcp-compare \
  fixtures/mcp/catalog-before.json \
  fixtures/mcp/catalog-after.json --json
pnpm --silent self-managed-api scan fixtures/consumer --json
```

La comparación MCP detecta dos cambios incompatibles y termina con código **1**, como está previsto. El escáner lee dos archivos y encuentra una llamada; no ejecuta código del consumidor. Estos dos comandos no requieren motor OpenAPI, Docker, credenciales ni inferencia. Los ejemplos con barra invertida al final de línea usan shell POSIX; en PowerShell reunir la orden en una línea.

Qué demuestra cada resultado

Salida	Límite
Diff estructural	No establece la semántica de negocio.
Impacto	Asocia usos posibles; exige revisión de cobertura.
Tests superados	Prueban sus aserciones, no todos los comportamientos.
PR en borrador	No equivale a merge, adopción o despliegue.

Comparar OpenAPI e instalar el SDK

El paquete `self-managed-api` es ESM y no está publicado en npm. Instalar desde el checkout o un archivo generado a partir de una revisión conocida. No instalar paquetes ajenos con nombres parecidos.

Motor OpenAPI explícito

```
pnpm engine:setup
pnpm --silent self-managed-api compare \
  fixtures/orders-v1.json fixtures/orders-v2.json \
  --engine work/tools/oasdiff/oasdiff --json
```

En Windows usar `work/tools/oasdiff/oasdiff.exe`. La fixture ficticia detecta dos cambios y termina con 1. `engine:setup` descarga oasdiff 1.33.0 desde su release oficial, verifica el SHA-256 fijado y conserva LICENSE. Necesita red y tar. Comparar no descarga el motor automáticamente.

compare devuelve **0** sin cambios estructurales incompatibles; **1** si los detecta; **2** ante entrada inválida o fallo del motor. Leer stderr JSON.

Otro proyecto consumidor

Este checkout fija minimatch 11.0.4 bajo @tufjs/models 4.1.1 por GHSA-g3hv-75wg-m249. La política raíz se aplica al checkout y sus builds CLI; no se hereda al instalar el tarball SDK. Antes de instalarlo, añadir una de estas políticas en la raíz de la aplicación receptora.

pnpm-workspace.yaml:

```
overrides:
  '@tufjs/models@4.1.1>minimatch': 11.0.4
```

package.json raíz para npm (combinar con sus campos existentes):

```
{"overrides":{"@tufjs/models@4.1.1":{"minimatch":"11.0.4"}}}
```

```
pnpm pack --pack-destination work/packages
# Desde la aplicación receptora; sustituir la ruta real:
pnpm add --ignore-scripts /ruta/self-managed-api-0.1.0.tgz
pnpm why minimatch
pnpm exec self-managed-api --help
```

Con npm usar install --ignore-scripts y ls minimatch. Verificar la relación @tufjs/models@4.1.1 → minimatch@11.0.4, conservar el lockfile y ejecutar los tests de la aplicación. Revisar esta política al actualizar TUF; otras copias requieren su propia revisión.

El packing compila, no publica ni incluye fixtures u oasdiff. Instalar offline requiere dependencias en el store local. Usar imports públicos, ESM/NodeNext y tipos de Node; no rutas privadas de src o dist.

Importar el SDK no abre una base de datos, arranca un worker, instala herramientas o ejecuta el consumidor. La página siguiente muestra un ejemplo SDK. Los portables de release incluyen runtime CLI; el SDK instalado en una aplicación mantiene el requisito Node.

Inspeccionar el impacto

```
self-managed-api impact base.json revision.json \
  --consumer /srv/application \
  --provider-base-url https://api.example.test/v1 \
  --engine /opt/self-managed-api/oasdiff --json
```

Las rutas y dominios de esta guía son ejemplos y deben sustituirse. OpenAPI admite JSON 3.0.x/3.1.x. El análisis JS/TS cubre patrones acotados de fetch nativo y algunas lecturas de respuesta. No equivale a comprender cualquier import, wrapper, SDK, alias o ruta dinámica. Los usos no resueltos y cambios sin correspondencia se conservan para revisión.

```
import { compareOpenApi } from 'self-managed-api';
const report = await compareOpenApi({
  basePath: '/contracts/before.json', revisionPath: '/contracts/after.json',
  enginePath: '/opt/self-managed-api/oasdiff',
});
```

Otros contratos y evidencias

Entrada	Contrato
MCP	Catálogo completo declarado y bindings explícitos de archivo, receiver, servidor y contexto autorizado. Comparar no invoca herramientas.
JSON Schema	Seleccionar dirección input/output. La conclusión se limita al perfil estructural admitido.
Ejemplos JSON	Detectan diferencias de valores. No establecen campos obligatorios ni compatibilidad.
Documentación	Evidencia de texto que puede incluir cambios cosméticos. No autoriza ejecutar código.
Python / Java	Fuentes y tests seleccionados explícitamente; sin descubrimiento automático de llamadas.

```
self-managed-api mcp-compare before.json after.json --json
self-managed-api schema-compare before.json after.json \
  --direction input --json
self-managed-api example-compare before.json after.json --json
```

Captura HTTPS opt-in

Registrar una política local no hace una petición. La observación requiere autorización explícita de red. La captura usa URL y origen HTTPS públicos exactos, puerto 443, sin credenciales en URL ni query/fragmento. Comprueba DNS y fija una dirección pública; rechaza redirecciones, compresión, destinos privados y cuerpos fuera de límites. No sigue enlaces ni descubre todas las versiones de un proveedor.

La política limita tamaño y tiempo. La autenticación de un proveedor privado se entrega mediante callbacks SDK locales de confianza; no se obtiene de la documentación remota ni se interpreta como una instrucción. La captura y sus credenciales no requieren ningún servicio del proyecto.

Registrar una dependencia

Una suscripción relaciona fuente, baseline aceptada y repositorio consumidor. Estado, snapshots, jobs y recibos viven en SQLite local. Guardarlos en una ruta privada, persistente y fuera del checkout consumidor. No compartir la misma base entre máquinas.

```
self-managed-api subscribe /contracts/current.json \
  --baseline /contracts/accepted.json \
  --consumer /srv/application \
  --provider-base-url https://api.example.test/v1 \
  --engine /opt/self-managed-api/oasdiff \
  --state /private/self-managed-api-state --json

self-managed-api worker --once \
  --state /private/self-managed-api-state \
  --engine /opt/self-managed-api/oasdiff --json
self-managed-api jobs --state /private/self-managed-api-state
```

Registrar u observar no aprueba tests, llama a un modelo ni publica una PR. El primer cambio compatible con el perfil produce evidencia y un trabajo revisable. Una nueva observación no cambia silenciosamente la baseline aceptada.

Observación acotada o supervisada

```
self-managed-api worker-loop \
  --state /private/self-managed-api-state \
  --engine /opt/self-managed-api/oasdiff \
  --cycles 10 --interval-ms 60000 --json
```

El loop es finito: un ciclo por defecto y de 1 a 1.000 con --cycles. No requiere una licencia ni contacta un control remoto. Para ejecución persistente usar el servicio nativo descrito más adelante. Los orígenes remotos se capturan solo cuando se permite explícitamente esa política.

Estado	Acción
unchanged / observed	Fuente igual o nueva evidencia registrada. No acredita compatibilidad del consumidor desplegado.
busy	Otro proceso tiene la concesión. No borrar locks ni arrancar escritores simultáneos.
decision-required	Resolver decisiones de negocio; no tratarlo como un parche ejecutable.
blocked / interrupted	Investigar evidencia y recuperar el intento antes de autorizar otra ejecución.

Snapshots y recibos conservan su timestamp de evidencia. Arrancar un proceso no refresca una observación histórica. Los informes de estado locales tampoco sustituyen la comprobación del proceso o de su manager.

Proponer sin ceder la decisión

Las recetas locales revisadas permiten planificar sin inferencia. Si se elige un modelo, el usuario prepara un endpoint compatible o sus credenciales de proveedor. El proyecto no incluye modelos, cuentas de proveedor ni consumo prepagado; tampoco instala pesos automáticamente.

```
self-managed-api job-proposal-plan JOB_ID \
  --state /private/self-managed-api-state \
  --engine /opt/self-managed-api/oasdiff \
  --model /private/model-connections.json \
  --test orders.test.mjs
```

El plan captura un contexto acotado. Inspeccionar destinos, archivos, pruebas, datos y límites antes de confirmar la solicitud exacta. Un modelo externo necesita consentimiento adicional para la salida de datos. Las instrucciones en documentación, comentarios o respuestas del modelo no pueden conceder permisos.

Resultados y consentimiento

Resultado	Qué permite
proposal-ready	Revisar receta, cambios y preguntas. No valida ni publica.
decision-required	Leer resumen y responder decisiones pendientes. No contiene una receta utilizable como migración.
Respuesta inválida o negativa	Falla explícitamente; no se convierte en éxito ficticio.

Manual es el modo predeterminado de conexión. Auto permite únicamente el fallback aprobado y acotado. Verificar una conexión realiza inferencia sintética; las verificaciones al arrancar requieren permiso --startup-checks separado. No hay polling de inferencia en reposo.

Revisión semántica

El revisor resuelve moneda, paginación, identidad, políticas de error u otras decisiones que un diff estructural no puede deducir. Las respuestas se ligan al digest de la propuesta capturada. Cambiar el consumidor, contexto o receta invalida la identidad revisada y exige generar e inspeccionar una nueva solicitud.

Los tests seleccionados son de solo lectura para el modelo. Una propuesta no puede modificar las expectativas de las pruebas para aparentar mejora. Python y Java capturan archivos con hashes y requieren revisar cobertura y selección de tests explícitamente.

Guardar secretos en archivos privados o en el mecanismo del proveedor elegido. No incluirlos en ejemplos públicos, recetas, capturas, logs ni control de versiones. GitHub y el modelo usan permisos independientes.

Aprobar y comprobar los mismos tests

Planificar, aprobar, encolar y ejecutar son acciones distintas. Cada aprobación se vincula a entradas exactas, tiene vigencia limitada y se consume una sola vez. Revisar el plan inmediatamente antes de aprobarlo. Cambiar archivos o parámetros exige nueva revisión.

```
self-managed-api job-plan JOB_ID \
  --state /private/self-managed-api-state \
  --engine /opt/self-managed-api/oasdiff \
  --recipe /private/reviewed-recipe.json --test orders.test.mjs
# Revisar la salida antes de job-approve y job-queue.
self-managed-api run-approved --once \
  --state /private/self-managed-api-state \
  --engine /opt/self-managed-api/oasdiff --json
```

La validación ejecuta original y candidato en copias aisladas distintas, sobre los mismos tests seleccionados. No modifica el consumidor vivo, no instala dependencias descubiertas ni autoriza publicación. La aprobación de validación caduca tras 15 minutos y no cubre merge o despliegue.

La CLI exige terminal interactivo y confirmación del identificador exacto de la solicitud. Las funciones de aprobación SDK confían en su llamador: no autentican por sí mismas a una persona. Una aplicación que las integre debe definir su propia revisión y autorización.

Host y perfiles

Perfil	Requisitos y límite
JavaScript / TypeScript	Linux x64, Docker y runtime Node fijado. No ejecuta scripts arbitrarios del package.json.
Python unittest	Python 3.13.16; módulos importables con casos seleccionados.
Java javac-main	Temurin Java 25; clases seleccionadas con assertions activadas.
pytest / Maven-JUnit	Runtimes preparados explícitamente. Pytest 9.1.1 o Maven 3.9.16/JUnit 6.1.3; solo perfil de dependencias documentado, no multimódulo arbitrario.

```
self-managed-api runtime prepare pytest --output /private/pytest.json
self-managed-api runtime prepare maven-junit --output /private/maven.json
```

La preparación descarga dependencias oficiales verificadas. La ejecución del consumidor permanece sin red y usa imagen/receta/dependencias ligadas al consentimiento. Pytest desactiva autoload de plugins; tests, conftest, POM y manifiestos Python permanecen inmutables.

Passed, failed e improved son resultados diferentes. Un baseline fallido y candidato correcto puede clasificarse improved y terminar con 1, no como éxito de ambos. Recuperar intentos interrumpidos con recover-approved; no repetir escrituras inciertas ni borrar marcadores para forzar verde.

Del resultado a una PR en borrador

La publicación es opcional y exige un repositorio privado en github.com, con permiso de escritura, activo y que no sea un fork. No admite destinos públicos ni GitHub Enterprise. Una aprobación de tests no concede escritura Git. El adaptador crea una rama y PR en borrador con el cambio exacto revisado; no hace merge ni despliega.

```
self-managed-api publication-plan INTENT_ID \
  --state /private/self-managed-api-state \
  --git /usr/bin/git --engine /opt/self-managed-api/oasdiff \
  --repository OWNER/REPO --base main --json
```

Este comando construye un candidato local ligado a Git. No contacta GitHub ni comprueba la base remota o la elegibilidad del destino; publication-review --github verifica ambas antes de aprobar. Git debe aceptar --no-lazy-fetch para evitar adquisiciones implícitas al inspeccionar objetos.

Permisos separados

1. Revisar el candidato y las diferencias exactas.
2. Preparar la solicitud de publicación mediante el adaptador GitHub, con identidad de autor explícita.
3. Confirmar publication-approve para esa solicitud y consumidor.
4. Ejecutar publication-publish con el recibo, la solicitud y --github.
5. Si el resultado es incierto, reconciliar la misma solicitud antes de intentar otra escritura.

publication-review, publication-publish y publication-reconcile requieren --github y GH_TOKEN configurado localmente. Conceder únicamente los permisos necesarios para el repositorio y sus PR. Nunca entregar el token en URLs, archivos de ejemplo o argumentos que se registren públicamente.

Código	Interpretación
0	Publicación verificada o resultado completado según el comando.
1	Conflicto que requiere revisión.
2	Incertidumbre, autorización inválida o fallo. No asumir que no hubo escritura.

Reconciliar no reintenta escrituras. Una pérdida de respuesta puede ocurrir después de crear la rama o PR. Conservar la evidencia y la identidad de la operación. El usuario sigue decidiendo revisión final, merge y despliegue.

Las demostraciones grabadas y fixtures de publicación simulada se identifican como tales. No constituyen una PR real ni prueban acceso a un repositorio ajeno.

Observación persistente

El servicio local funciona sin cuenta, licencia remota ni servidor de control. Su manager nativo se ocupa del arranque y reinicio. El servicio observa fuentes registradas y conserva estado; no genera autorizaciones ni ejecuta cambios por su cuenta.

```
{"schemaVersion":1,
 "statePath":"/var/lib/self-managed-api/state",
 "allowHttp":false,
 "openApi":{"enginePath":"/opt/self-managed-api/oasdiff",
            "intervalMs":60000},
 "sources":{"intervalMs":900000}}

self-managed-api worker-service run \
  --config /etc/self-managed-api/worker.json
self-managed-api worker-service status \
  --config /etc/self-managed-api/worker.json
self-managed-api worker-service stop \
  --config /etc/self-managed-api/worker.json
```

Quitar perfiles sin suscripciones; al menos uno es obligatorio. Configuración schemaVersion 1 de hasta 64 KiB, campos cerrados, rutas absolutas y literales, sin hooks ni secretos inline. models.configPath es opcional. Permitir capturas HTTPS de forma explícita; cambiar configuración exige reinicio revisado.

systemd

```
self-managed-api worker-service render \
  --config /etc/self-managed-api/worker.json \
  --target systemd --node /usr/bin/node \
  --cli /opt/self-managed-api/installed-cli.js \
  --user managedapi --name managedapi
```

El comando produce una definición para revisar; no la instala. Resolver la CLI instalada mediante import.meta.resolve('self-managed-api/cli'). La unidad lanza Node directamente, sin shell o pnpm. Preparar cuenta no-root, consumidor/config de solo lectura y estado privado escribible.

Scope system es el predeterminado. Scope user omite --user y necesita manager activo, namespaces de usuario/montaje y delegación cgroup v2 de CPU, memoria y procesos. Los límites Linux configurados son una CPU, MemoryHigh 384 MiB, MemoryMax 512 MiB, sin swap y TasksMax 128. Comprobar aplicación efectiva en el host; no inferirla de la presencia del archivo.

status es un snapshot durable y no prueba que el PID siga vivo. Consultar también el manager y la cadencia real de observación. Un proceso colgado puede requerir intervención; un restart no sustituye un watchdog.

Operar el servicio en cada host

macOS y Windows

macOS utiliza launchd con un LaunchDaemon y cuenta dedicada. Windows genera XML para WinSW 2.12.0, provisionado y verificado por el operador, ejecutado como LocalService. Otros servicios que usan esa identidad comparten sus permisos. Aplicar ACL solo a las rutas necesarias; instalar servicios de sistema requiere privilegios administrativos.

```
self-managed-api worker-service render --config C:\private\managedapi\worker.json --target windows --node C:\tools\node.exe --cli C:\tools\self-managed-api\dist\cli.js --name managedapi
```

En JSON Windows usar barras invertidas escapadas y rutas con letra de unidad. No se expanden variables ni rutas ambiguas. Los límites cgroup de Linux no se presentan como garantías para Windows o macOS.

Parada y sustitución

1. Revisar versión, firmas/checksums y requisitos antes de preparar un paquete nuevo.
2. Solicitar stop y comprobar el estado detenido en el manager. En Windows esperar Stopped: WinSW puede retornar mientras SCM informa StopPending.
3. Conservar estado, configuración y evidencias privadas. No mover una base SQLite activa entre hosts.
4. Sustituir únicamente archivos de instalación revisados y arrancar con la misma configuración compatible.
5. Comprobar proceso, cadencia de observación y resultado local. Ante fallo, detener y recuperar con los archivos compatibles conservados.

Las configuraciones y snapshots anteriores con campos de control remoto o licencia se rechazan. Para migrar, detener primero la versión anterior mediante systemctl, launchctl o SCM, revisar la configuración local admitida y arrancar la nueva versión. No borrar bases o evidencias para eludir incompatibilidades. El nuevo arranque válido escribe su propio snapshot.

La parada se liga al runId; una orden vieja no detiene el proceso de reemplazo. El manager permite hasta 180 segundos de drenaje antes de forzar cierre. Los locks de proceso y de SQLite se conservan hasta terminar; no borrar bases, índices o marcadores para sortear un bloqueo.

Estado y permisos

Guardar datos bajo una cuenta dedicada y aplicar cifrado/backups del entorno: SQLite no aporta cifrado de aplicación. Estado corrupto devuelve error; no se reinicializa silenciosamente. Logs locales sanitizados y acotados no sustituyen la retención y observabilidad del sistema operativo.

En Linux, un fallo 226/NAMESPACE exige revisar política del host o usar un scope compatible, conservando restricciones. systemd 249 puede requerir delegación CPU adicional para scope user. No conceder root al worker para evitar investigar la configuración.

Preparar un paquete verificado

El updater admite un repositorio TUF público configurado por el usuario. No necesita cuenta ni token del proyecto. No incluye una raíz o feed oficial incorporado. También es válido actualizar manualmente desde un archivo revisado, sin usar el updater.

```
{"statePath":"/private/self-managed-api-updates",
 "repository":{
   "metadataUrl":"https://releases.example.test/metadata/",
   "targetsUrl":"https://releases.example.test/targets/",
   "rootPath":"/private/release-root.json",
   "rootSha256":"HUELLA_SHA256_VERIFICADA_POR_OTRO_CANAL"}}

self-managed-api client-update check --config /private/updates.json
self-managed-api client-update stage --config /private/updates.json
self-managed-api client-update status --config /private/updates.json
```

La configuración anterior es ilustrativa: URLs, archivo y huella deben corresponder al repositorio real. La confianza inicial se obtiene mediante un canal independiente del servidor de descarga. No confiar automáticamente en una raíz y su huella recibidas del mismo origen no verificado.

Contrato

TUF verifica firmas/umbrales, caducidad, rotación de raíz, rollback y hash/longitud del target. El canal channels/stable.json conserva schemaVersion 1, versión x.y.z y secuencia creciente. El target SDK es packages/self-managed-api/x.y.z/self-managed-api-x.y.z.tgz. No se aceptan instrucciones remotas de instalación.

check refresca metadatos; stage descarga solo un paquete más nuevo; status lee y valida estado local sin red. La salida downloaded-awaiting-approval siempre conserva installation: not-authorized. No extrae archivos, ejecuta scripts, instala npm, cambia el runtime ni reinicia servicios.

El transporte exige HTTPS público sin credenciales/query/fragmento, sin redirecciones ni compresión. Usa resolución DNS comprobada y conexión TLS. Metadatos/paquetes tienen límites de tamaño, 40 peticiones, 40 MiB y deadline compartido de 120 segundos. El SDK está limitado a 32 MiB; hasta 16 rotaciones de raíz por operación.

Conservar la caché de confianza. Cambiar bytes con la misma identidad o borrar la base para eludir un rollback es inválido. Ante expiración, el operador renueva metadatos firmados; una firma no demuestra que el código carezca de fallos. Los paquetes descargados siguen siendo datos hasta la instalación explícita.

Construir y exportar una versión

```
pnpm release:build
pnpm release:sign --config /absolute/operator/release.json
pnpm release:refresh --config /absolute/operator/refresh.json
```

release:build usa el Node 24.21.0-24.x del builder para un ejecutable SEA y lib materializada, motor oasdiff, licencias y PDF. Necesita dependencias preparadas y red para obtener la licencia exacta de Node. Verifica el archivo extraído fuera del checkout, incluyendo help y comparaciones reales. Extraer todo el directorio portable.

Windows sin certificado queda expresamente sin editor verificado; macOS sin identidad usa firma ad hoc, no notarización. Los recibos solo afirman verificación cuando el verificador del SO confirma la identidad configurada. Las variables SELF_MANAGED_API_WINDOWS_* y SELF_MANAGED_API_APPLE_* son configuración de operador; las credenciales/certificados no se aprovisionan desde una PR.

TUF público: entrada y custodia

release:sign recibe schemaVersion 1, rootFile/rootSha256, roleKeys, version, sequence, sourceRevision, artifacts, documentationFile, notes y requirements (es/en), y metadataVersions opcional. Los archivos y rutas son absolutos; las claves Ed25519 privadas se leen bajo permisos restrictivos y no se exportan.

La salida privada conserva metadata/, targets/, release-bundle.json y recibos nativos públicos. Solo repository/ es el árbol de transporte público. Con consistent_snapshot true contiene raíz numerada, targets/snapshot versionados y targets prefijados por hash; no publicar alias mutables. Guardar el directorio de operador completo fuera del árbol servido.

Renovar sin reconstruir

release:refresh recibe schemaVersion, releaseDirectory, rootSha256, manifestSha256 y roleKeys. Autentica metadatos históricos incluso expirados, comprueba todos los targets y exige una raíz aún vigente. Conserva código, manifiesto, canal y raíz byte a byte. Renueva timestamp una hora; snapshot un día y targets siete días cuando sea necesario, sin superar la raíz.

Una nueva versión del producto debe usar versiones de metadatos superiores a todas las renovaciones anteriores. Publicar primero targets/snapshot y finalmente timestamp. El comando solo crea una nueva generación local; no publica el feed ni instala paquetes.

Publicación manual

Ejecutar los comandos desde un checkout revisado con dependencias fijadas. El operador revisa los archivos y publica manualmente la versión y, si lo utiliza, el repositorio TUF. La firma requiere claves e identidades propias; conservar separadas la configuración privada y los archivos públicos.

Verificar el proyecto

Comando	Requisitos y resultado
pnpm check	Typecheck, suite ordinaria, laboratorio de modelos sin inferencia obligatoria y paquete instalado en consumidor offline.
pnpm test:isolated	Docker Linux x64 e imágenes/runtimes preparados explícitamente. Ejecuta validación de consumidores aislados.
pnpm docs:build	Dependencia Playwright local y su Chromium. Genera el PDF y comprueba geometría de todas las páginas.
pnpm docs:check	Comprueba hash de HTML, versión y número de páginas; no requiere otro repositorio.

```
pnpm engine:setup
pnpm check
pnpm exec playwright install chromium
pnpm docs:build
pnpm docs:check
```

Ejecutar las comprobaciones localmente. La validación necesita Linux x64; las comprobaciones de supervisión nativa corresponden a cada sistema operativo y mantienen scope user. Preparar las imágenes fijadas antes de ejecutar validaciones: el runtime usa --pull=never. No reducir tests, cobertura o aislamiento para aprobar un cambio.

Laboratorios opcionales

dev:local prepara una API de pagos ficticia, un consumidor y contratos editables bajo work/. Su Compose sirve únicamente el laboratorio local, no es un requisito del SDK. dev:apis observa documentación pública de proveedores y reproduce transportes simulados. Un proveedor observado no acredita migraciones completas para todas sus operaciones.

dev:polyglot comprueba recetas Python/Java seleccionadas. Los laboratorios de modelos requieren pesos/runtime propios y consentimiento explícito; sus resultados no prueban calidad general de migraciones. Los datos sintéticos y la publicación simulada deben seguir identificados como tales.

Dependencias y datos

oasdiff y sus licencias pertenecen al motor externo. Babel parser sostiene inspección sintáctica; tuf-js sostiene el updater público. Node, imágenes y dependencias de pruebas conservan sus propias licencias y requisitos. La licencia MIT del proyecto no sustituye sus atribuciones.

Mantener .env, estado, claves, pesos, propuestas, recibos y logs completos fuera de Git. work/ es la ubicación ignorada para intermediarios del proyecto; no publicar esos archivos como documentación o artefactos de release. Revisar historial y contenido preparado antes de hacer pública una versión.

Fuente técnica: docs/technical-guide.html. El PDF identifica versión del paquete y SHA-256 de la fuente. Un cambio de contrato exige actualizar ambos, renderizar y revisar todas las páginas. Los resultados de validación de una tarea pertenecen a la conversación, no a esta guía.